

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#02)

Implementation of Combinational Circuits using Verilog Gate & Dataflow Level Modeling

Student Name: RUQAIYA ASAD Roll No: 24I-6519 Section: CE-A

Date performed: 25/8/2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

Half Adder: two bits are used ,XOR gives the sum and AND gives the carry, gate level uses xor and and ,while dataflow uses ^ and &.**Full Adder:** three bits A, B and Cin are used ,gate level builds it using two half adders and an or gate, while dataflow uses ^ for sum and & with | for carry.

Code:

```
module ha_gate(sum, carry, a, b);

    input a, b;

    output sum, carry;

    xor (sum, a, b);

    and (carry, a, b);

endmodule

module ha_dataflow(sum, carry, a, b);

    input a, b;

    output sum, carry;

    assign sum = a ^ b;

    assign carry = a & b;

endmodule

module fa_gate(sum, carry, cin, a, b);

    input a, b, cin;

    output sum, carry;
```

```
wire t1, t2, t3;

// First Half Adder

ha_gate HA1(t1, t2, a, b);

// Second Half Adder

ha_gate HA2(sum, t3, t1, cin);

// OR the two carry outputs

or (carry, t2, t3);

endmodule

module fa_dataflow(sum, carry, cin, a, b);

    input a, b, cin;

    output sum, carry;

    assign sum  = a ^ b ^ cin;

    assign carry = (a & b) | (b & cin) | (a & cin);

endmodule

module testbench;

    reg A, B, Cin;

    wire hs_gate, hc_gate;

    wire hs_dataflow, hc_dataflow;
```

```
wire fs_gate, fc_gate;

wire fs_dataflow, fc_dataflow;

ha_gate    HA_GATE    (hs_gate,    hc_gate,    A, B);

ha_dataflow HA_DATAFLOW (hs_dataflow, hc_dataflow, A, B);

fa_gate    FA_GATE    (fs_gate,    fc_gate,    Cin, A, B);

fa_dataflow FA_DATAFLOW (fs_dataflow, fc_dataflow, Cin, A, B);


initial begin

    A = 0; B = 0; Cin = 0;


    #10 A = 0; B = 0; Cin = 1;

    #10 A = 0; B = 1; Cin = 0;

    #10 A = 0; B = 1; Cin = 1;

    #10 A = 1; B = 0; Cin = 0;

    #10 A = 1; B = 0; Cin = 1;

    #10 A = 1; B = 1; Cin = 0;

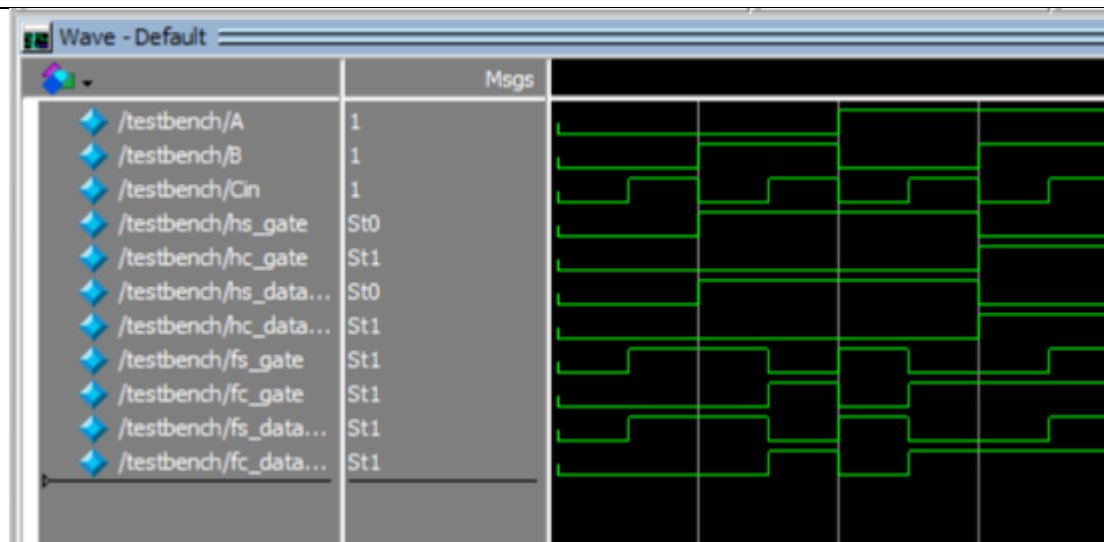
    #10 A = 1; B = 1; Cin = 1;

    #10 $finish;

End

endmodule
```

Output:



Task 2:

Description:

It has 3 input pins and 8 output pins. Depending on the binary value of the inputs, exactly one output becomes 1 and the rest stay 0. In gate level we use not gates to invert inputs and and gates to pick the correct output line. In dataflow we write the same logic using assign statements with ~ and &.

Code:

```
// 3x8 DECODER - GATE LEVEL

module decoder_gate(y0, y1, y2, y3, y4, y5, y6, y7, a, b, c);

    input a, b, c;
```

```
output y0, y1, y2, y3, y4, y5, y6, y7;

wire a_n, b_n, c_n;

not (a_n, a);

not (b_n, b);

not (c_n, c);

and (y0, a_n, b_n, c_n);

and (y1, a_n, b_n, c);

and (y2, a_n, b, c_n);

and (y3, a_n, b, c);

and (y4, a, b_n, c_n);

and (y5, a, b_n, c);

and (y6, a, b, c_n);

and (y7, a, b, c);

endmodule

// 3x8 DECODER _dataflow LEVEL

module decoder_dataflow(y0, y1, y2, y3, y4, y5, y6, y7, a, b, c);

    input  a, b, c;

    output y0, y1, y2, y3, y4, y5, y6, y7;

    assign y0 = ~a & ~b & ~c;

    assign y1 = ~a & ~b & c;

    assign y2 = ~a & b & ~c;
```

```
assign y3 = ~a & b & c;
```

```
assign y4 = a & ~b & ~c;
```

```
assign y5 = a & ~b & c;
```

```
assign y6 = a & b & ~c;
```

```
assign y7 = a & b & c;
```

```
endmodule
```

```
module testbench();
```

```
    reg a, b, c;
```

```
    wire g0, g1, g2, g3, g4, g5, g6, g7;
```

```
    wire d0, d1, d2, d3, d4, d5, d6, d7;
```

```
    decoder_gate uut1(g0, g1, g2, g3, g4, g5, g6, g7, a, b, c);
```

```
    decoder_dataflow uut2(d0, d1, d2, d3, d4, d5, d6, d7, a, b, c);
```

```
    initial
```

```
    begin
```

```
        a = 0; b = 0; c = 0;
```

```
        #50 a = 0; b = 0; c = 1;
```

```
        #50 a = 0; b = 1; c = 0;
```

```
        #50 a = 0; b = 1; c = 1;
```

```
        #50 a = 1; b = 0; c = 0;
```

```
        #50 a = 1; b = 0; c = 1;
```

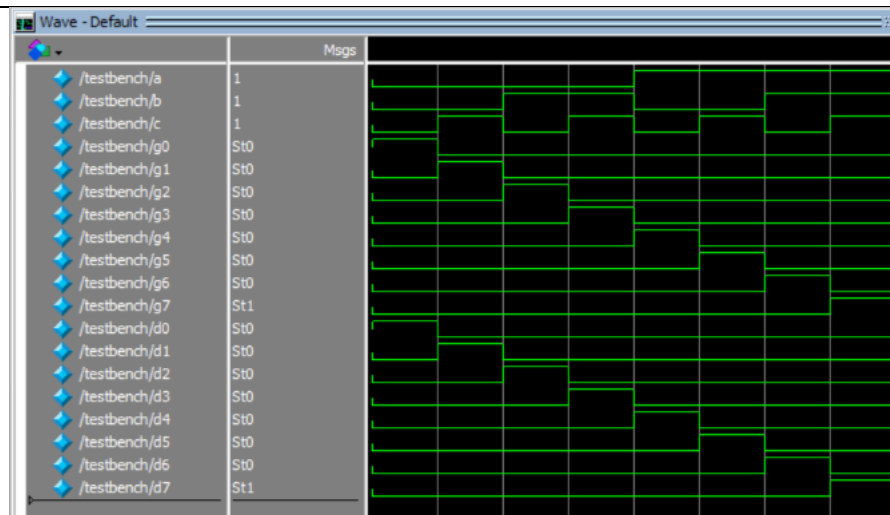
```
        #50 a = 1; b = 1; c = 0;
```

```
#50 a = 1; b = 1; c = 1;
```

```
end
```

```
endmodule
```

Output:



Task 3:

Description:

It has 8 data inputs, 3 select lines and 1 output. The select lines decide which data input gets sent to the output. In gate-level we use not gates for the inverted select bits, and gates to enable each data input and one or gate to combine them all. In dataflow we write one big statement using ~, &, and |.

Code:

```
// 8x1 MUX - GATE LEVEL
```

```
module mux_gate(y, d0, d1, d2, d3, d4, d5, d6, d7, s2, s1, s0);
```

```
    input d0, d1, d2, d3, d4, d5, d6, d7;
```

```
    input s2, s1, s0;
```

```
    output y;
```



```
wire s2_n, s1_n, s0_n;

wire t0, t1, t2, t3, t4, t5, t6, t7;

not (s2_n, s2);

not (s1_n, s1);

not (s0_n, s0);

and (t0, d0, s2_n, s1_n, s0_n);

and (t1, d1, s2_n, s1_n, s0);

and (t2, d2, s2_n, s1, s0_n);

and (t3, d3, s2_n, s1, s0);

and (t4, d4, s2, s1_n, s0_n);

and (t5, d5, s2, s1_n, s0);

and (t6, d6, s2, s1, s0_n);

and (t7, d7, s2, s1, s0);

or (y, t0, t1, t2, t3, t4, t5, t6, t7);
```

```
endmodule
```

```
// 8x1 MUX - DATAFLOW LEVEL
```

```
module mux_dataflow(y, d0, d1, d2, d3, d4, d5, d6, d7, s2, s1, s0);
```

```
    input d0, d1, d2, d3, d4, d5, d6, d7;
```

```
    input s2, s1, s0;
```

```
output y;

assign y = (~s2 & ~s1 & ~s0 & d0) |

    (~s2 & ~s1 & s0 & d1) |

    (~s2 & s1 & ~s0 & d2) |

    (~s2 & s1 & s0 & d3) |

    ( s2 & ~s1 & ~s0 & d4) |

    ( s2 & ~s1 & s0 & d5) |

    ( s2 & s1 & ~s0 & d6) |

    ( s2 & s1 & s0 & d7);

endmodule

module testbench();

    reg d0, d1, d2, d3, d4, d5, d6, d7;

    reg s2, s1, s0;

    wire y_gate;

    wire y_dataflow;

    mux_gate uut1(y_gate, d0, d1, d2, d3, d4, d5, d6, d7, s2, s1, s0);

    mux_dataflow uut2(y_dataflow, d0, d1, d2, d3, d4, d5, d6, d7, s2, s1, s0);

    initial

    begin

        // Select d0 (s2 s1 s0 = 000)
```

```
d0=1; d1=0; d2=0; d3=0; d4=0; d5=0; d6=0; d7=0;

s2=0; s1=0; s0=0;

// d1 (s2 s1 s0 = 001)

#50 d0=0; d1=1; d2=0; d3=0; d4=0; d5=0; d6=0; d7=0;

s2=0; s1=0; s0=1;

// d2 (s2 s1 s0 = 010)

#50 d0=0; d1=0; d2=1; d3=0; d4=0; d5=0; d6=0; d7=0;

s2=0; s1=1; s0=0;

// d3 (s2 s1 s0 = 011)

#50 d0=0; d1=0; d2=0; d3=1; d4=0; d5=0; d6=0; d7=0;

s2=0; s1=1; s0=1;

// d4 (s2 s1 s0 = 100)

#50 d0=0; d1=0; d2=0; d3=0; d4=1; d5=0; d6=0; d7=0;

s2=1; s1=0; s0=0;

// d5 (s2 s1 s0 = 101)

#50 d0=0; d1=0; d2=0; d3=0; d4=0; d5=1; d6=0; d7=0;

s2=1; s1=0; s0=1;
```

```
// d6 (s2 s1 s0 = 110)

#50 d0=0; d1=0; d2=0; d3=0; d4=0; d5=0; d6=1; d7=0;

    s2=1; s1=1; s0=0;

// d7 (s2 s1 s0 = 111)

#50 d0=0; d1=0; d2=0; d3=0; d4=0; d5=0; d6=0; d7=1;

    s2=1; s1=1; s0=1;

end
```

endmodule

Output:

